CAPÍTULO 2 · SISTEMAS DISTRIBUIDOS Y CONTENEDORES · SESIÓN 4

Redes, almacenamiento y autoscaling en inferencia

Diseño de Infraestructura Escalable para LLMs · Maestría · 3 horas

Resultados de aprendizaje



Diseñar la ruta de red

Justificar L4/L7, algoritmos de balanceo, timeouts, streaming y segmentación Zero Trust para tráfico de LLM.



Dimensionar el almacenamiento

Calcular el tamaño de pesos y KV cache; elegir objeto/bloque/archivo/NVMe y reducir el cold start.



Definir una política de autoscaling

Elegir señales, umbrales y ventanas; estimar capacidad con la ley de Little.



Planificar GPU

Comparar device plugin, time-slicing, MIG, MPS y DRA; controlar la colocación y la fragmentación.

Agenda y distribución del tiempo

Bloque	Contenido	Min
1	Redes: balanceo, streaming, backpressure, Zero Trust	35
2	Almacenamiento: pesos, KV cache, cold start, volúmenes	30
—	Descanso	10
3	Autoscaling: señales, Little, estabilidad, escala a cero	35
4	GPU scheduling: device plugin, MIG, DRA, colocación	25
5	Demostraciones en vivo (Ollama, cola, 429, KEDA)	30
6	Laboratorios multicloud, discusión y cierre	15

Caso transversal: asistente institucional

5 000

usuarios potenciales (supuesto)

100

usuarios concurrentes iniciales (supuesto)

99,9 %

disponibilidad objetivo ≈ 43 min/mes de indisponibilidad

N+1

redundancia mínima para tolerar la caída de una réplica

Requisito	Consecuencia de diseño en esta sesión
Acceso autenticado y datos protegidos	Red privada, egress controlado, identidad sin claves
Restricción presupuestal	Escalar por demanda real; mínimo y máximo de réplicas
Trazabilidad y observabilidad	Métricas de cola, TTFT y rechazos como SLIs

Idea central: capacidad finita, demanda variable



Red
reparte y protege

Almacenamiento
modelo listo

Autoscaling
ajusta la oferta

GPU scheduling
asigna lo escaso

01

Redes para inferencia

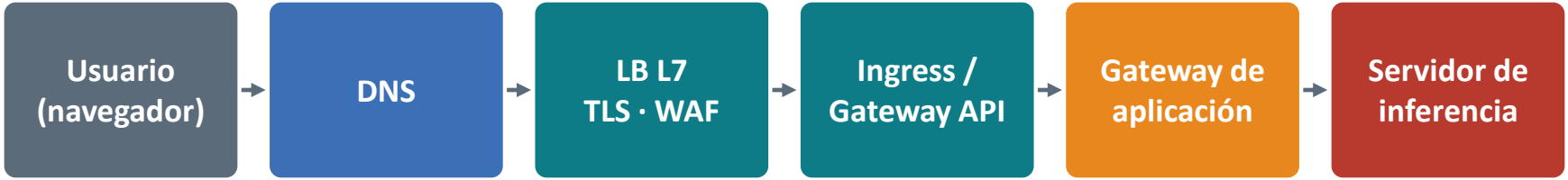
Balanceo · streaming · backpressure · segmentación Zero Trust

Una solicitud LLM no es una solicitud web

Dimensión	API web tradicional	Generación con LLM
Duración	milisegundos	segundos a minutos
Costo por solicitud	casi uniforme	proporcional a tokens de salida (desconocido)
Conexión	corta	larga, con streaming de tokens (SSE)
Recurso limitante	CPU / E/S	VRAM (KV cache) y cómputo de GPU
Efecto de la saturación	latencia sube gradualmente	cola crece; aparecen rechazos y timeouts

Fuente conceptual: Kwon et al., PagedAttention (SOSP 2023); Yu et al., Orca (OSDI 2022).

La ruta de red de una pregunta al modelo



Salto	Qué controla	Fallo típico
LB L7	TLS, WAF, límites de tasa, idle timeout	corta generaciones largas
Ingress	enrutamiento, buffering, timeouts	buffering rompe SSE
Gateway de aplicación	autenticación, admisión, métricas	cola sin límite
Servidor de inferencia	batching, KV cache	VRAM agotada → rechazos

Balanceo L4 frente a L7

L4 — transporte

- Decide por IP y puerto
- Reparte conexiones
- Baja latencia, bajo costo
- Ciego a rutas, cabeceras y cuerpo
- Riesgo: conexiones persistentes fijan al cliente a una réplica

L7 — aplicación

- Entiende HTTP
- Reparte solicitudes
- TLS, WAF, límites de tasa, reintentos
- Enruta por ruta, cabecera o modelo
- Costo y latencia algo mayores

Algoritmos de balanceo y su señal

Algoritmo	Reparte según	Límite en LLM
Round-robin	turno	ignora el costo de cada generación
Least-connections	conexiones activas	una conexión no equivale a una unidad de trabajo
EWMA / latencia	latencia reciente	señal con retraso
Consciente del modelo	cola, KV cache, prefijo	requiere métricas del servidor y un endpoint picker

Hecho documentado: GKE Inference Gateway + Gateway API Inference Extension (kubernetes-sigs). Ver referencias.

Repartir conexiones no es repartir trabajo



Unidades de tiempo de generación (ilustrativo): A recibió las generaciones largas; B termina y queda ociosa.

Síntoma

p99 alto con CPU y GPU promedio «normales».

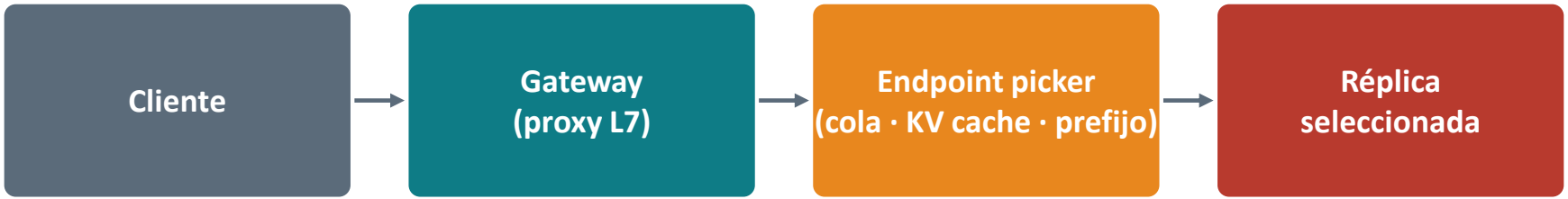
Remedio

Enrutar por cola/KV cache y limitar max_tokens.

Streaming (SSE), timeouts y buffering

Trampa	Síntoma	Control
Buffering del proxy	tokens llegan «de golpe»	desactivar buffering para la ruta SSE
Idle/read timeout bajo	corte a mitad de respuesta	timeout > generación máxima; limitar max_tokens
Conexiones persistentes con L4	una réplica saturada, otra libre	balanceo L7 por solicitud
Reintentos automáticos	amplifican la sobrecarga	reintentar solo con backoff y presupuesto de reintentos

Balanceo consciente del modelo



Cola y KV cache

Elige la réplica con más capacidad real.

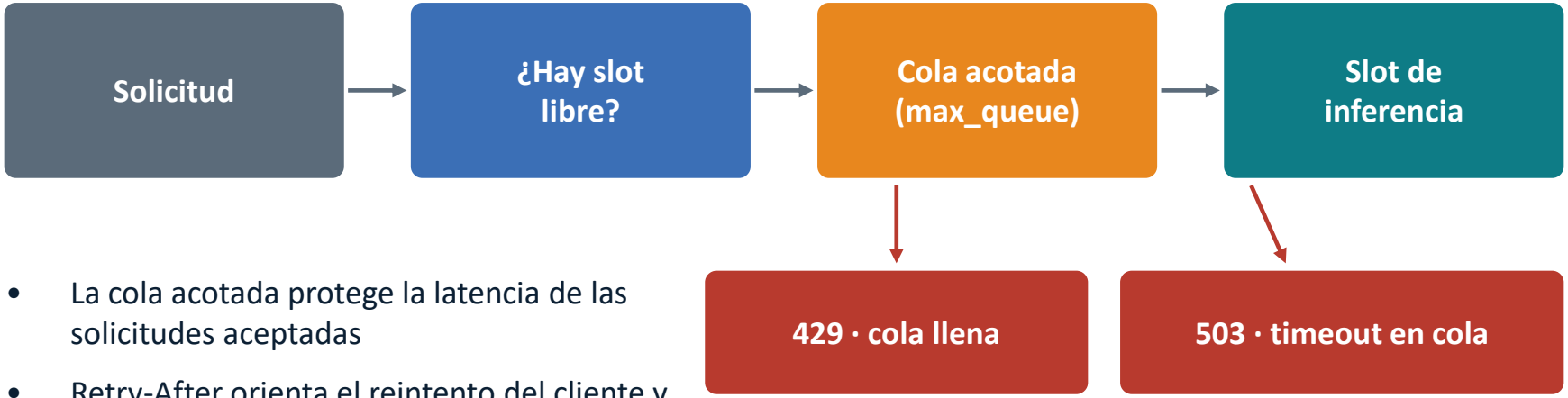
Prefijo en caché

Reutiliza prefill de prompts comunes.

Prioridad

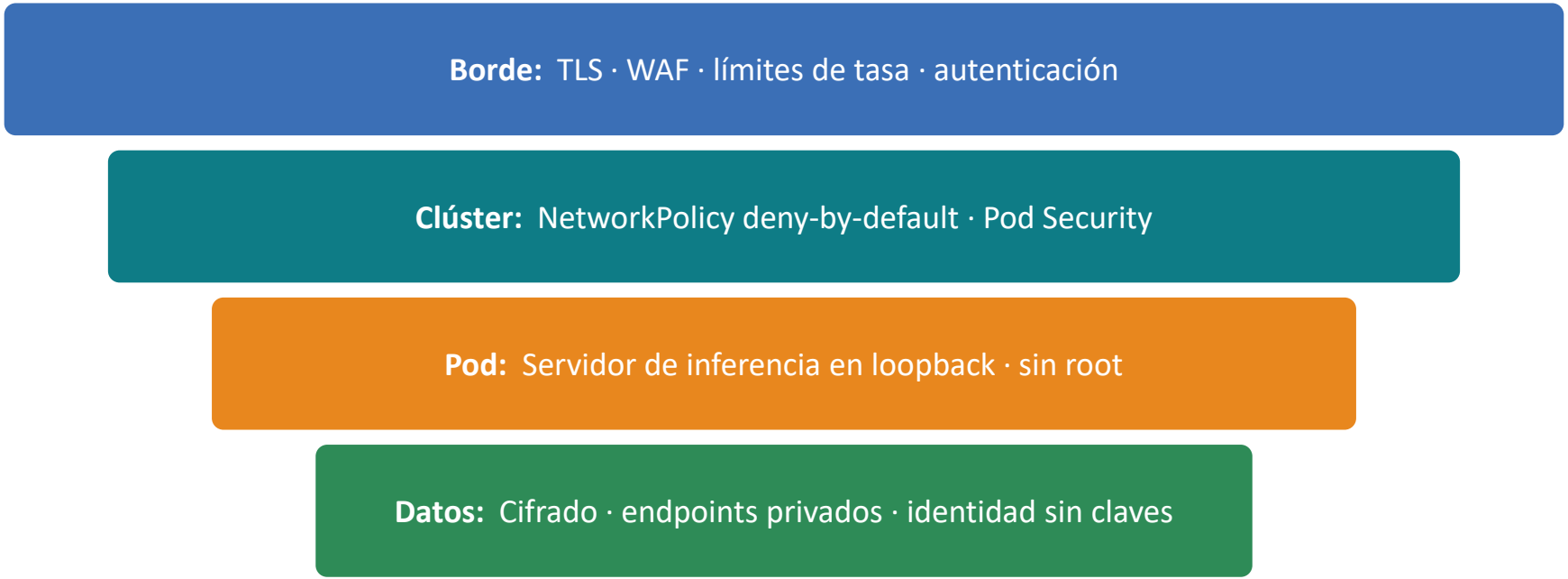
Interactivo antes que lotes.

Backpressure y load shedding



- La cola acotada protege la latencia de las solicitudes aceptadas
- Retry-After orienta el reintento del cliente y evita tormentas de reintentos
- Métricas: queue_depth, rejected_total, queue_wait

Segmentación y Zero Trust de red



Requiere un CNI con soporte de NetworkPolicy (Calico, Cilium, Azure CNI + NPM/Cilium, GKE Dataplane V2…).

Egress, endpoints privados y SSRF

Amenaza	Vector	Control
Exfiltración	egress abierto	deny-by-default + allowlist
Robo de credenciales	SSRF a 169.254.169.254	bloquear rango de metadatos; identidad de carga de trabajo
Cadena de suministro	modelo/paquete no verificado	fuente privada + checksum + versión fija
Fuga por la API pública	tráfico a Internet	Private Endpoint / PrivateLink / PSC

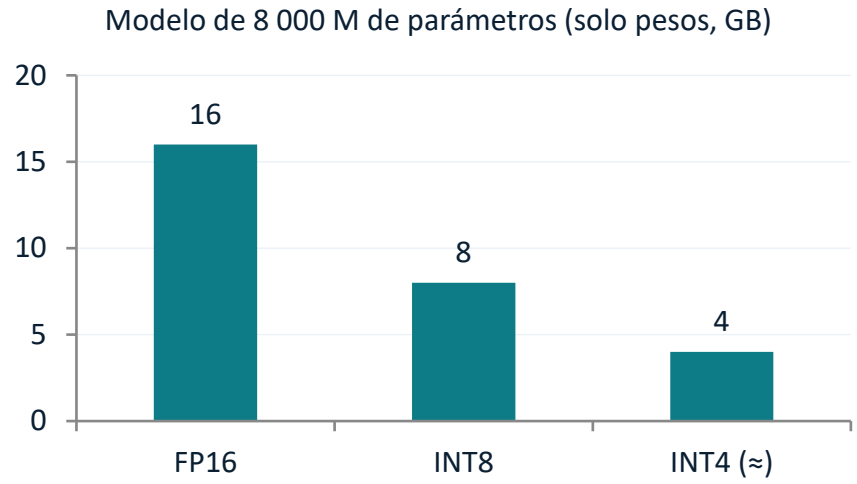
02

Almacenamiento para inferencia

Pesos · KV cache · cold start · volúmenes · gobierno de modelos

El tamaño de los pesos es aritmética

bytes de pesos ≈ parámetros × bytes por parámetro



Parámetros	FP16	INT8	INT4 (≈)
1 000 M	2 GB	1 GB	0,5 GB
8 000 M	16 GB	8 GB	4 GB
70 000 M	140 GB	70 GB	35 GB

Cálculo teórico solo de pesos; sin KV cache ni activaciones.

La memoria total: pesos + KV cache + activaciones

KV cache ≈ 2 × capas × cabezas KV × dim. cabeza × bytes × tokens



Pesos

constantes por réplica; se cargan una vez



KV cache

crece con contexto × concurrencia



Activaciones

temporales durante el cómputo

VRAM disponible para KV cache = VRAM total − pesos − activaciones − margen. Ese valor limita la concurrencia por réplica.

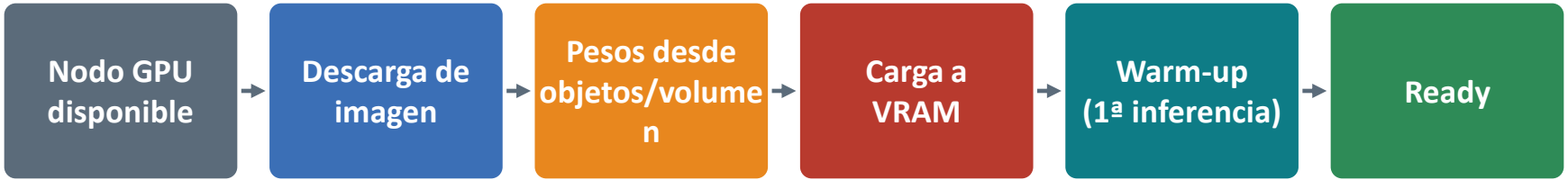
Fuente: Kwon et al., «Efficient Memory Management for LLM Serving with PagedAttention», SOSP 2023.

Tipos de almacenamiento y su rol

Tipo	Rol	Ventaja	Riesgo
Objetos	fuente de verdad versionada	durable, barato, políticas	descarga lenta → cold start
Bloque (RWO)	volumen por réplica	baja latencia	atado a zona y a un Pod
Archivos (RWX)	caché compartida	una copia para N réplicas	rendimiento variable, costo
NVMe local	caché rápida	máxima velocidad	se pierde con el nodo

Anatomía del cold start de una réplica

$$T_{listo} = T_{nodo} + T_{imagen} + T_{pesos} + T_{VRAM} + T_{warm\text{-}up}$$



Sin cifras: dependen del proveedor, el nodo, el modelo y la red. Mídalas en su laboratorio (Pending → Ready).

- Cold start largo ⇒ más réplicas mínimas y más margen
- Cada término tiene su palanca (ver siguiente diapositiva)

Palancas para reducir el cold start

Nodo

Sobreaprovisionar con Pods «pause» desalojables

Imagen

Pequeña, sin pesos, registro cercano

Pesos

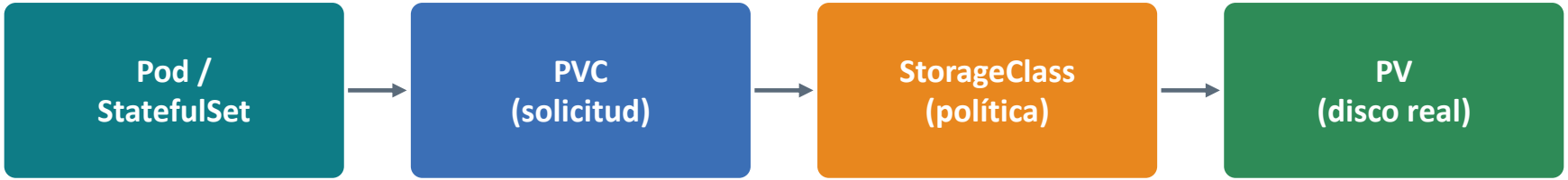
Volumen poblado o caché NVMe

VRAM y warm-up

KV-alive alto y solicitud de calentamiento

Todo precalentamiento cuesta capacidad ociosa: compárelo con el costo de incumplir el SLO.

Volúmenes en Kubernetes: PV, PVC y StorageClass



Modo	Significado	Uso en inferencia
ReadWriteOnce	un nodo a la vez	un volumen por réplica (StatefulSet)
ReadWriteMany	varios nodos	caché compartida de pesos
ReadOnlyMany	varios, solo lectura	pesos inmutables y versionados

Los volúmenes de bloque son zonales: alta disponibilidad multizona exige réplicas con su propio volumen en cada zona.

Gobierno de modelos: versión, integridad, cifrado

Versionado

inmutable y trazable por réplica

Integridad

checksum o firma antes de cargar

Cifrado

en reposo y en tránsito; claves del cliente si aplica

Acceso

mínimo privilegio sobre repositorios de modelos

Marcos: NIST AI RMF 1.0 y AI 600-1 (Perfil IA Generativa); ISO/IEC 42001:2023.

Descanso · 10 minutos

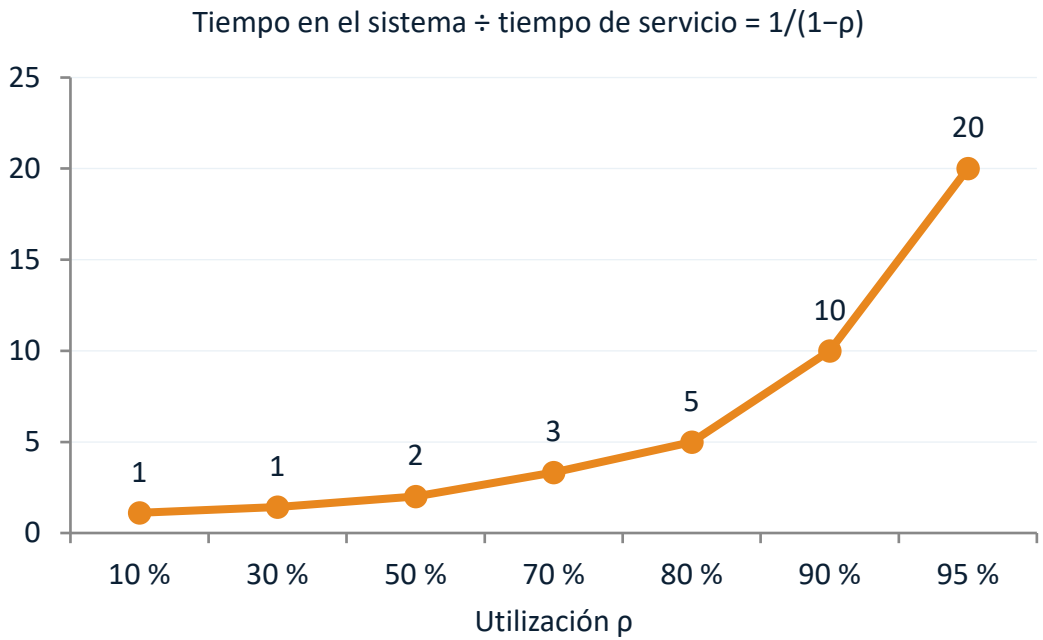
Al volver: autoscaling. Piense en qué señal escalaría usted y por qué.

03

Autoscaling de inferencia

Señales · ley de Little · estabilidad · escala a cero

Utilización y latencia: la curva de palo de hockey



Lección

Cerca del 100 % la espera se dispara.

Diseño

Utilización objetivo + margen + cola acotada.

Ley de Little: cuántas solicitudes hay en el sistema

$$L = \lambda \cdot W$$

Símbolo	Significado	Ejemplo supuesto (caso)
λ	tasa de llegada (solicitudes/s)	5 /s si cada usuario encadena solicitudes sin pausa
W	tiempo en el sistema (s)	20 s por generación
L	solicitudes en el sistema	L = 5 × 20 = 100 generaciones simultáneas (caso extremo)
think time	pausa del usuario	con 40 s de lectura → ≈ 33 generaciones simultáneas

Todos los valores son supuestos académicos; la concurrencia que dimensiona la flota es la de GENERACIÓN, no la de usuarios.

Señales de autoscaling: de peor a mejor

Señal	Qué mide	Veredicto para LLM
CPU / memoria	uso del nodo	engañosa: satura de inmediato o no refleja la cola
Utilización de GPU	ocupación del acelerador	alta incluso con carga sana; no distingue saturación
Profundidad de cola	demanda insatisfecha	señal principal: anticipa la degradación
KV cache / TTFT	presión de memoria y síntoma	guardrail: llega cuando el usuario ya sufre

Hecho documentado: Amazon EKS — «Autoscale AI inference with HPA and KEDA» usa cola y p95 e2e como disparadores.

Escalamiento horizontal, vertical y de clúster



Horizontal

HPA / KEDA
Cambia el número de réplicas
Principal mecanismo
Límites: cold start y cuota de GPU



Vertical

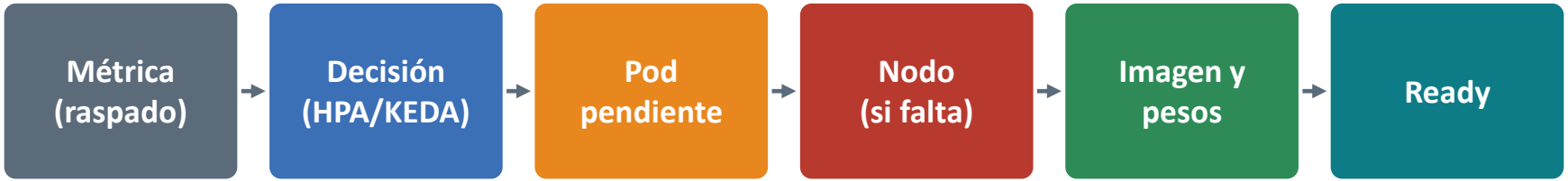
VPA / ajuste de requests
Cambia CPU y RAM por Pod
No cambia la VRAM de una GPU
Suele reiniciar el Pod



De clúster

Cluster Autoscaler / Karpenter
Agrega nodos GPU
Añade minutos al camino crítico
Exige cuota

La cadena de latencia del escalado



segundos → minutos

- Mientras escala: la cola acotada y el margen sostienen el servicio
- Picos predecibles (horario de clases): escalado programado
- Nunca confiar solo en el reactivo para picos más cortos que la cadena

Estabilidad: evitar el flapping

Técnica	Efecto	Parámetro del repositorio
Subida rápida	responde antes a la demanda	+2 Pods / 60 s, sin ventana
Bajada lenta	conserva capacidad caliente	−1 Pod / 120 s, ventana 300 s
Umbral de activación	ignora ruido	activationThreshold = 1
Mínimo y máximo	SLO y presupuesto	min 1 · max 4

Los umbrales son supuestos iniciales: se calibran con la prueba de carga (loadtest.py).

Escalar a cero: ahorro frente a latencia



Ventajas

- Sin costo en reposo
- Ideal para lotes y evaluaciones
- Combina bien con nodos spot



Costos

- Primera solicitud paga el cold start completo
- Inaceptable para chat interactivo
- Requiere cola o respuesta asíncrona

Autoscaling en las tres nubes: dos rutas

	Ruta A: Kubernetes administrado	Ruta B: serverless / endpoints administrados
Azure	AKS + KEDA / HPA	Container Apps · Foundry (modelos administrados)
AWS	EKS + KEDA / Karpenter	ECS/Fargate · Bedrock (modelos administrados)
GCP	GKE + HPA / Inference Gateway	Cloud Run · Vertex AI (modelos administrados)
Control	alto, portable	menor; escala el proveedor
Operación	más trabajo del equipo	menos trabajo; límites y cuotas del proveedor

Verifique cada servicio y capacidad GPU en la documentación vigente: cambian con frecuencia.

04

Planificación de GPU

Device plugin · time-slicing · MIG · MPS · DRA · colocación

Por qué la GPU es un recurso especial

Entera y exclusiva

Se pide por unidades enteras; sin sobreasignación por defecto.

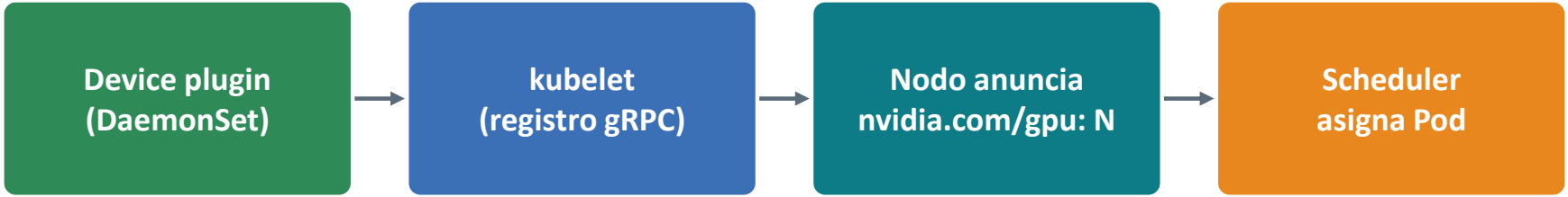
Topología

NVLink/PCIe importan para paralelismo tensorial.

Escasa y cara

Pocos nodos, cuota limitada, cargas en competencia.

Device plugin: cómo Kubernetes descubre la GPU

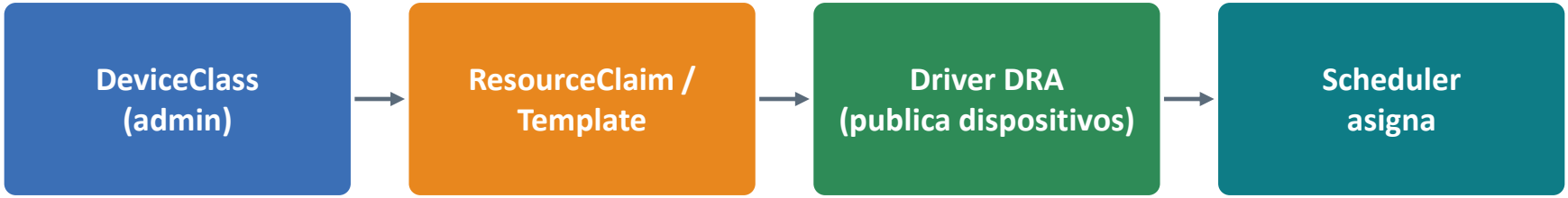


- El scheduler solo ve un contador entero
- Sin atributos (modelo, VRAM), sin compartición fina
- Aislamiento de pools con taints y tolerations

Compartir una GPU: time-slicing, MIG y MPS

Mecanismo	Idea	Aislamiento	Cuándo
GPU entera	un Pod, una GPU	total	producción con SLO
Time-slicing	turnos sobre una GPU	sin aislamiento de memoria	desarrollo, modelos pequeños
MIG	particiones físicas	fuerte	multiinquilino; hardware compatible
MPS	comparte cómputo	parcial	muchos procesos pequeños

DRA: reclamar dispositivos como almacenamiento



Qué aporta

Selección por atributos (modelo, VRAM), configuración por reclamo y base para compartición fina.

Estado (hecho)

resource.k8s.io/v1 GA en Kubernetes 1.34; driver NVIDIA donado a la comunidad (mar-2026).

Colocación, fragmentación y tipos de carga



Dónde

nodeSelector, afinidad, taints/tolerations



Topología

varias GPU del modelo en el mismo nodo/NVLink



Fragmentación

GPU parciales que bloquean cargas grandes



Tipos de carga

en línea dedicada; lotes en spot con colas

FinOps: el costo por millón de tokens

$$USD/1M\ tokens = USD_GPU_hora \div (tok/s \times 3600 \times utilización) \times 10^6$$

Utilización

la palanca principal: una GPU ociosa cobra igual

Tamaño y cuantización

modelo suficiente para la tarea

Modalidad de compra

reservada para la base, spot para lotes

Sin precios en el repositorio: use los vigentes de su región y sus propias mediciones.

Checklist de seguridad de la sesión

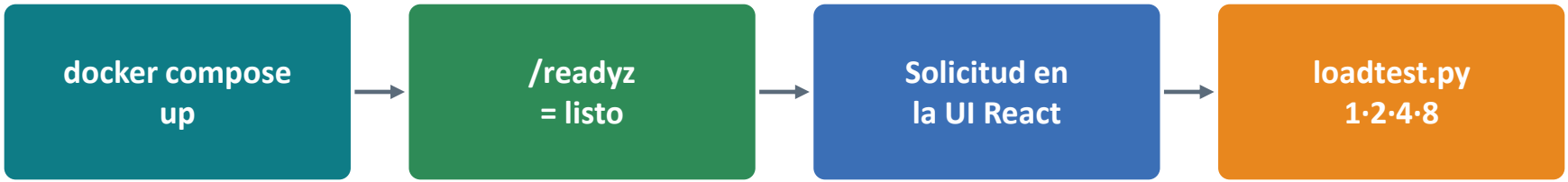
Dominio	Verificación mínima
Red	deny-by-default · TLS · inferencia no expuesta · egress mínimo
Almacenamiento	cifrado · versión fija · checksum · acceso mínimo
Autoscaling	máximo de réplicas · cola acotada · límites por usuario (OWASP LLM10:2025)
GPU	aislamiento según riesgo (entera / MIG) · pools reservados
Identidad	sin claves estáticas · identidad de carga de trabajo

05

Demostraciones en vivo

Predicciones primero · datos después

Demo 1 — Ollama + gateway: observar la cola



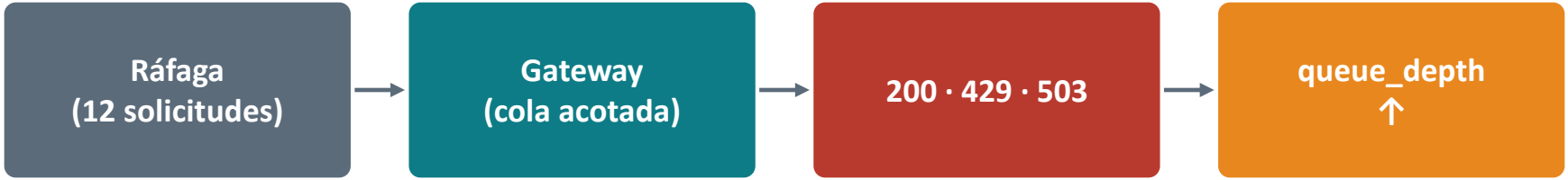
Predicción (escribirla antes)

¿Cómo crece la latencia p95 al aumentar la concurrencia?

Qué observar

queue_depth, in_flight, TTFT, TPOT y 'codes' del reporte.

Demo 2 — Ráfaga: 200, 429 y 503



- ¿Un error rápido y claro o una espera de minutos?
- Con KEDA: queue_depth alto ⇒ nueva réplica ⇒ medir Pending → Ready
- Lo que no se mide en clase no se afirma como resultado

Laboratorios multicloud: un agente, tres nubes

Google Cloud

Consola · ADK · Cloud Run / Agent Engine · Gemini

Microsoft Azure

Portal Foundry · Agent Framework · SK · AutoGen · LangGraph

AWS

Consola Bedrock · Strands · AgentCore Runtime

Verificado: importaciones y construcción de objetos. NO verificado: llamadas reales a modelos en nube (ejecutar antes de clase).

Modelos y regiones cambian: confirme IDs vigentes (p. ej., Gemini 2.5 retirado en Vertex AI en oct-2026 según fuentes consultadas).

Decisiones arquitectónicas del caso (ADR)

ADR	Decisión	Trade-off principal
001	Gateway con cola acotada por réplica	un componente más; límite por réplica
002	Autoscaling por profundidad de cola (KEDA)	depende de Prometheus y de calibrar el umbral
003	StatefulSet con un PVC por réplica	almacenamiento duplicado; primer arranque lento
004	NetworkPolicy deny-by-default; inferencia en loopback	requiere CNI; egress 443 es deuda a eliminar
005	Identidad sin claves estáticas	más pasos iniciales de configuración
006	Ollama en el laboratorio	no representa el rendimiento de servidores de alto throughput

Discusión guiada

1. ¿Por qué dos réplicas con el mismo número de conexiones pueden estar desigualmente cargadas?

2. ¿Por qué rechazar tráfico (429) puede mejorar la disponibilidad percibida?

3. Con un cold start de minutos, ¿qué combinación de mecanismos protege un pico de 30 s?

4. ¿Qué mecanismo de compartición de GPU elegiría para un clúster multiusuario y por qué?

Síntesis: cinco ideas para recordar

1. La capacidad es finita: admitir con cola acotada y rechazar con claridad.
2. Balancear trabajo, no conexiones: L7 y, a escala, consciente del modelo.
3. Los pesos son datos pesados: el cold start fija la velocidad máxima de escalado.
4. Escalar por cola (demanda insatisfecha), con estabilidad, mínimos y máximos.
5. La GPU se planifica: entera, MIG, time-slicing o DRA según aislamiento y costo.

Actividad de evaluación y entregables

Laboratorio 0

Mediciones propias con loadtest.py

Informe técnico

Capacidad, autoscaling, red y almacenamiento

Laboratorios 1–3

Comparación de al menos dos frameworks

Autoevaluación

Cuestionario de la guía del estudiante

Referencias (1/2) — Fundamentos

- Kwon, W. et al. (2023). Efficient Memory Management for LLM Serving with PagedAttention. SOSP '23.
- Yu, G.-I. et al. (2022). Orca: A Distributed Serving System for Transformer-Based Generative Models. OSDI '22.
- Zhong, Y. et al. (2024). DistServe: Disaggregating Prefill and Decoding for Goodput-optimized LLM Serving. OSDI '24.
- Dean, J. y Barroso, L. A. (2013). The Tail at Scale. Communications of the ACM, 56(2).
- Little, J. D. C. (1961). A Proof for the Queuing Formula: L = λW. Operations Research, 9(3).
- Beyer, B. et al. (2016). Site Reliability Engineering. O'Reilly.

Referencias (2/2) — Documentación y marcos

- Kubernetes: Dynamic Resource Allocation; Device Plugins; HPA; Network Policies — kubernetes.io/docs.
- KEDA — keda.sh/docs (ScaledObject, disparador Prometheus).
- Google Cloud: GKE Inference Gateway; blog «DRA: A new era of Kubernetes device management» (25-mar-2026).
- AWS: Amazon EKS — Autoscale AI inference with HPA and KEDA; Amazon Bedrock AgentCore (GA 13-oct-2025).
- Microsoft: Agent Framework 1.0 (blog Foundry, abr-2026); documentación de Microsoft Foundry.
- OWASP Top 10 for LLM Applications 2025 · NIST AI RMF 1.0 y AI 600-1 · ISO/IEC 42001:2023 · ISO/IEC 27001:2022.
- FinOps Foundation — FinOps Framework.

Preguntas

Próxima sesión: observabilidad y pruebas de carga con SLO sobre este mismo sistema.